

Análisis a la Complejidad Computacional

Prof. Diana Margot López Herrera¹, Prof. Javier Fernando Botía Valderrama²,
Prof. Juan Felipe Quintana³

¹ Docente Principal de Lógica y Representación II

² Asesor Temático para la materia Lógica y Representación II

³ Docente de Cátedra para la materia Lógica y Representación II

Facultad de Ingeniería. Departamento de Ingeniería de Sistemas



1 Propiedades de las Notaciones Asintóticas

En el anterior documento titulado *Introducción a la Complejidad Computacional*, se definió la notación asintótica como una representación matemática definida por funciones polinómicas para determinar la eficiencia o no de un algoritmo. Considerando la diversidad de notaciones asintóticas para analizar la complejidad computacional de un algoritmo, es necesario conocer un conjunto de propiedades para facilitar la relaciones entre notaciones Big $\mathcal{O}$, Big Ω , Big Θ , entre otras.

En primer lugar, la **simetría** es una propiedad que permite relacionar la cota superior e inferior. Para su representación matemática, se utilizará la expresión *si y solo si* como $\Leftrightarrow$ (tal y como se trabaja en lógica proposicional):

- $f(n) = \mathcal{O}(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
- $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

Esta propiedad indica que si $f(n) \leq c(g(n))$ entonces su *equivalencia* sería $g(n) \geq c(f(n))$ (Es exactamente la misma interpretación de la dualidad que se estudia en lógica proposicional, donde afirmaban que la ley de DeMorgan se representa como $\neg(p \vee q) = \neg p \wedge \neg q$ y su dual es $\neg(p \wedge q) = \neg p \vee \neg q$). La misma notación se aplica para la notación o y ω .

Otra propiedad importante es la **transitividad**. Para esta propiedad, se consideran tres funciones polinómicas, $f(n)$, $g(n)$ y $h(n)$, tal que se cumplen cinco casos:

- $[f(n) = \Theta(g(n)) \wedge g(n) = \Theta(h(n))] \Rightarrow f(n) = \Theta(h(n))$
- $[f(n) = \mathcal{O}(g(n)) \wedge g(n) = \mathcal{O}(h(n))] \Rightarrow f(n) = \mathcal{O}(h(n))$
- $[f(n) = \Omega(g(n)) \wedge g(n) = \Omega(h(n))] \Rightarrow f(n) = \Omega(h(n))$
- $[f(n) = o(g(n)) \wedge g(n) = o(h(n))] \Rightarrow f(n) = o(h(n))$
- $[f(n) = \omega(g(n)) \wedge g(n) = \omega(h(n))] \Rightarrow f(n) = \omega(h(n))$

El símbolo $\Rightarrow$ significa *implica que*. Analizando la propiedad transitiva, es una analogía de la ley del Silogismo Hipotético de la lógica proposicional, $p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$.

Conociendo la propiedad simétrica y transitiva, se establece la **regla de las tasas polinómicas**:

- Dado dos funciones polinómicas, $f(n)$ y $g(n)$, si se considera $f(n) = \Theta(g(n))$, entonces se puede afirmar que existen dos constantes, c_1 y c_2 tal que $c_1g(n) \leq f(n) \leq c_2g(n)$ y n tiende al infinito. Con la condición de desigualdad, se puede expresar una regla expresada como una razón entre $f(n)$ y $g(n)$:

$$c_1 \leq \frac{f(n)}{g(n)} \leq c_2 \quad (1)$$

Explicación de la regla:

La regla de las tasas polinómicas asume que un algoritmo se ejecuta en un tiempo $f(n) = \Theta(g(n))$, el cual experimentalmente se mide el tiempo de cómputo y posteriormente, se divide $\frac{f(n)}{g(n)}$. Si estamos en lo cierto sobre el tiempo de ejecución $f(n) = \Theta(g(n))$, la tasa $\frac{f(n)}{g(n)}$ debería estar confinada en un intervalo entre las constantes c_1 y c_2 . Por consiguiente, la tasa debería aplanarse cuando n es lo suficiente grande aunque puede generar algunas fluctuaciones entre c_1 y c_2 .

Es importante considerar la tasa $\frac{f(n)}{g(n)}$ a menudo converge a un simple valor constante diferente a cero cuando se realiza mediciones experimentales del costo de computo. No obstante, si se realiza una estimación matemática diferente a una notación Big Ω , en vez de obtener una convergencia a valor constante diferente a cero, se va obtener lo contrario, es decir, una convergencia a cero. En este caso, si $f(n) = o(g(n))$, la tasa converge a cero para valores grandes de n y si $f(n) = \omega(g(n))$, la tasa crecerá sin límites, generando una divergencia.

Ejemplo 1:

Una forma gráfica para analizar la regla de las tasas polinómicas es comparando el rendimiento de un algoritmo. Si asumimos dos funciones polinómicas, $f(n) = 3n^2 + 7n + 4$ y $g(n) = 11n^3$, la regla de las tasas polinómicas estaría dado por:

$$c_1 \leq \frac{3n^2 + 7n + 4}{11n^3} \leq c_2$$

Como $n > 0$, entonces $f(n) = o(g(n))$ y por consiguiente, se deberá observar una convergencia a cero a medida que crece n

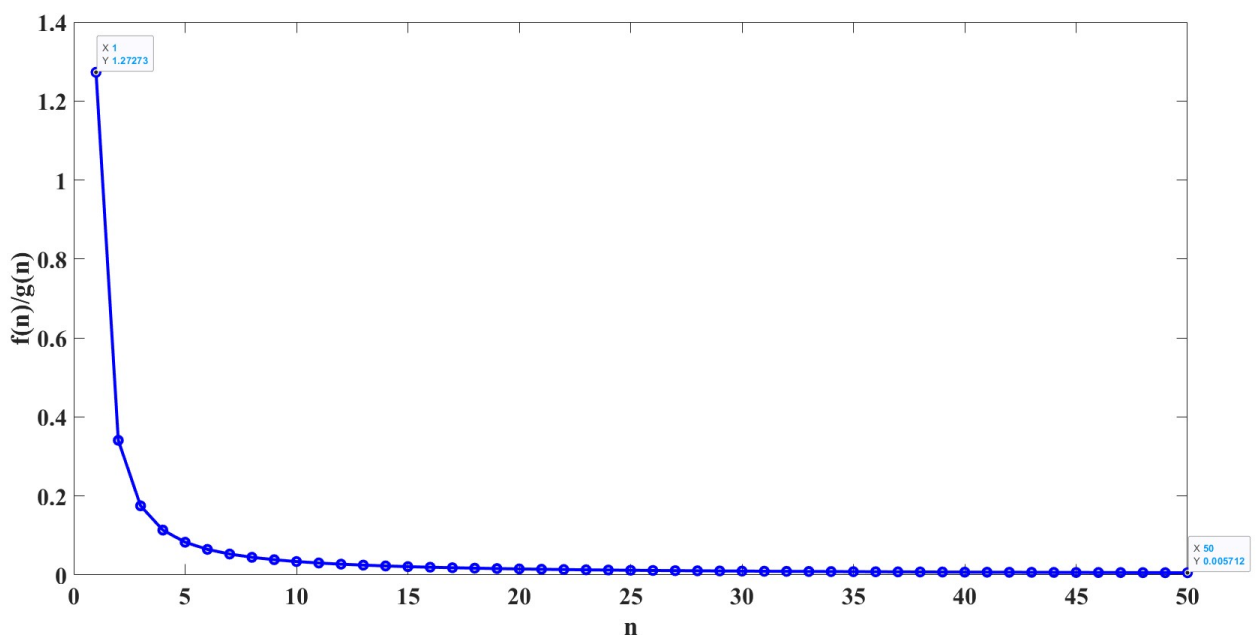


Figure 1: Tasa $\frac{3n^2+7n+4}{11n^3}$

En la Fig. 1, se puede determinar dos constantes 1.27273 y 0.005 y la siguiente regla:

$$0.005 \leq \frac{3n^2 + 7n + 4}{11n^3} \leq 1.27273; \quad \forall n > 0$$

Ejemplo 2:

Dado tres funciones polinómicas, $f(n) = 23n^2$, $g(n) = 42n^2 - 100n$, y $h(n) = 5247n^2$, determinar si se cumple con la propiedad $[f(n) = \mathcal{O}(g(n)) \wedge g(n) = \mathcal{O}(h(n))] \Rightarrow f(n) = \mathcal{O}(h(n))$.

Inicialmente, se analiza $23n^2 \leq \mathcal{O}(42n^2 - 100n)$, determinando para que valores n se cumple esta condición. Si se genera una gráfica comparativa entre $f(n)$ y $g(n)$, se encuentra que para valores $n \geq 6$, se cumple con la condición de la notación Big $\mathcal{O}$. Por otra parte, la condición $42n^2 - 100n \leq \mathcal{O}(5247n^2)$ se cumple para valores $n \geq 0$. Por consiguiente, si las dos condiciones anteriores se cumplen para la notación Big $\mathcal{O}$, se puede inferir que $23n^2 \leq \mathcal{O}(5247n^2)$ se cumple para valores $n \geq 0$, estableciendo una transitividad de $f(n)$ a $h(n)$ si ambas funciones cumple con la notación Big $\mathcal{O}$. El anterior análisis se puede visualizar en la Fig. 2:

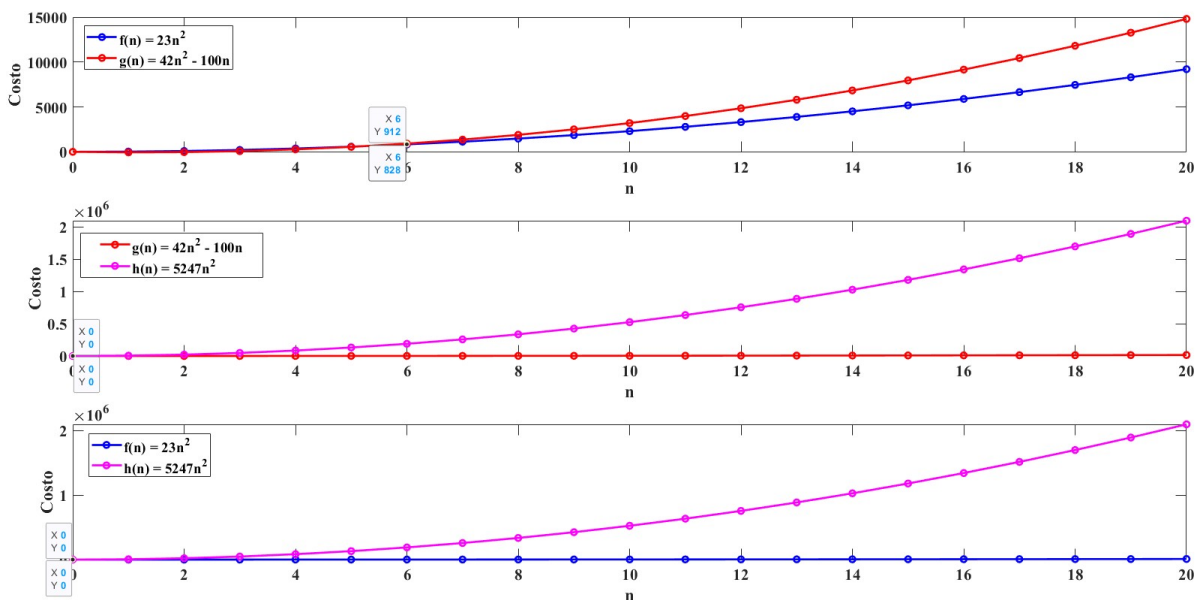


Figure 2: Gráficas de las funciones polinómicas $f(n) = 23n^2$, $g(n) = 42n^2 - 100n$, y $h(n) = 5247n^2$

Nota: Recuerden que la notación Big $\mathcal{O}$ debe satisfacer $\exists c, n_0 \mid f(n) \leq c(g(n))$.

2 Análisis Algorítmico y Aritmética de la Notación Big $\mathcal{O}$

2.1 Contador de frecuencias y las reglas de la suma aritmética y de acotamiento

Para analizar la eficiencia de un algoritmo, es necesario encontrar una expresión algebraica que represente la cantidad de ejecuciones de un conjunto de instrucciones del algoritmo. Lo anterior se le conoce como **contador de frecuencias**, el cual es una representación polinómica que se ha trabajado a lo largo del tema de la complejidad computacional. Veamos un primer caso definido como pseudocódigo y código de Python:

```
a ← 5
x ← a + 2
y ← x * a
write(y)
```

```
1 a = 5 # 0(1)
2 x = a + 2 # 0(1)
3 y = x*a # 0(1)
4 print(y) # 0(1)
```

En este algoritmo, cada secuencia se trabaja en notación Big $\mathcal{O}$. Como cada secuencia se ejecuta una sola vez, podemos realizar una sencilla suma de todas las secuencias:

$$\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(1) = \mathcal{O}(4)$$

Como obtenemos un valor constante igual a 4, el tiempo polinómico del algoritmo es *constante*, es decir, $\mathcal{O}(1)$.

Ahora veamos otro algoritmo:

```

 $n \leftarrow 100$ 
 $k \leftarrow 1$ 
for  $i \leftarrow 1, n$  do
     $k \leftarrow k + 1$ 
end for
write( $k$ )

```

```

1 n = 100 # O(1)
2 k = 1 # O(1)
3 for i in range(n): # O(n)
4     k = k + 1 # O(n)
5 print(k) # O(1)

```

Debido a que la sintaxis de Python es compacto para las operaciones *if - else*, *while* y *for*, no se coloca el comando *end()* para indicar finalización de la ejecución. Observando la secuencia del algoritmo, las líneas 1 y 2 se ejecutan una sola vez pero las líneas 4 y 5 se ejecuta n veces hasta cumplir con 100 ciclos ($n = 100$). Por consiguiente:

$$\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(1) = \mathcal{O}(2n + 3)$$

A partir de este resultado, se puede afirmar que el algoritmo se ejecuta en un tiempo *lineal* puesto que $\mathcal{O}(2n + 3) \approx \mathcal{O}(n)$

¿Qué sucede si cambiamos un ciclo *for* por un *while* en el anterior código?

```

 $n \leftarrow 100$ 
 $k \leftarrow 1$ 
 $i \leftarrow 1$ 
while  $i \leq n$  do
     $k \leftarrow k + 1$ 
     $i \leftarrow i + 1$ 
end while
write( $k$ )

```

```

1 n = 100 # O(1)
2 k = 1 # O(1)
3 i = 1 # O(1)
4 while i <= n: # O(n + 1)
5     k = k + 1 # O(n)
6     i = i + 1 # O(n)
7 print(k) # O(1)

```

En este algoritmo, se adiciono la línea 3 para representar el valor de partida de la variable i de tal forma que la instrucción *while* se ejecute hasta $j = 100$. Por tal motivo, dentro de la instrucción *while* se colocar un contador de la variable i para ejecute exactamente como en el algoritmo anterior, donde hay un ciclo *for*. Es importante mencionar que la línea 4 se ejecuta una vez más ya que cuando $i < n$, de todas maneras se hace la pregunta de si i es menor o igual a n . Teniendo en cuenta lo anterior, la costo computacional del algoritmo se expresa como:

$$\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(n + 1) + \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(1) = \mathcal{O}(3n + 5)$$

El costo computacional $\mathcal{O}(3n + 5) \approx \mathcal{O}(n)$, lo cual el algoritmo se ejecuta en tiempo *lineal*.

A partir de los tres ejemplos de análisis, es importante entender ¿por qué se calcula la complejidad computacional desde la suma de los costos de cada línea del algoritmo?. En este caso, la respuesta se encuentra en la teoría de la composición de funciones:

Regla de la suma aritmética de los costos computacionales:

Dado una función polinómica, f , y una constante, c , el producto $c(f)$ genera una nueva función h , tal que:

$$h(n) = c(f(n)) \quad (2)$$

La ecuación 2 esta definido para un conjunto de valores n tal que $\forall n \geq 0$. Ahora, si asumimos una suma de una función y una constante, $f + c$, se genera una nueva función $h(n) = f(n) + c$. Siguiendo esta notación, para el caso de dos funciones, f y g , si se construye una nueva función $h(n) = f(n)g(n)$, entonces, es valido afirmar que se puede construir una nueva función como la suma de f y g :

$$h(n) = f(n) + g(n) \quad (3)$$

La ecuación 3 indica una composición ($\circ$) de dos funciones, f y g , en una nueva función h , ya que $(f \circ g)(n) = f(g(n)) = h(n)$. Por consiguiente, la *regla de la suma aritmética* esta dado:

$$\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(f + g) \quad (4)$$

A partir de la regla de la suma aritmética, es importante entender porque acotamos la función polinómica a uno de los tiempos polinómicos de la complejidad computacional, es decir, por ejemplo $\mathcal{O}(3n + 5) \approx \mathcal{O}(n)$. En este caso, se establece una regla extra:

Regla del acotamiento:

Dado las funciones polinómicas, f_1 , f_2 y g , se puede relacionar estas funciones mediante la notación Big $\mathcal{O}$:

$$[f_1(n) = \mathcal{O}(g(n)) \wedge f_2(n) = \mathcal{O}(g(n))] \Rightarrow f_1(n) + f_2(n) = \mathcal{O}(g(n)) \quad (5)$$

La ecuación 5 es válido $\forall n \geq 0$. A partir de la anterior ecuación, cuando se suman dos funciones polinómicas del algoritmo, *siempre existirá una función dominante a la otra*. La dominancia de la función esta relacionado con la propiedad de la transitividad visto en la sección 1, razón por la cual el costo computacional se aproxima al *peor caso*. Por ejemplo, si $\mathcal{O}(8n) + \mathcal{O}(5n \log(n))$ entonces se puede aproximar al peor caso $\mathcal{O}(n \log(n))$ debido a que una tiempo log-lineal crece más rápido que un tiempo lineal para valores grandes de n .

Regla del producto aritmético de los costos computacionales

Realizando una versión análoga de la regla de la suma aritmética, se puede construir una nueva función h a partir de f y g mediante un producto:

$$h = \mathcal{O}(f) \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g) \quad (6)$$

Usualmente, la regla del producto es útil para analizar bucles y ciclos de los algoritmos. En este caso, el tiempo que se tarda en ejecutar un ciclo en n elementos es por lo general $\mathcal{O}(n)$ multiplicado por el tiempo que se tarda en ejecutar todo el bucle o ciclo, que podría ser una función $f(n)$:

$$n \cdot f(n) = \mathcal{O}(n \cdot f(n)) \quad (7)$$

La ecuación 7 se utiliza para estimar el costo de un contador dentro de un bucle principal, o un ciclos anidados, o sentencias *if-else* dentro de un bucle o ciclo principal. De la anterior ecuación, se puede definir la siguiente transformación matemática:

$$f \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g) \quad (8)$$

La ecuación 8 nos permite expresar el costo computacional como $n \times \mathcal{O}(1)$ cuando un ciclo *for* tiene un acumulador o contador o sentencias *if-else*. Veamos el siguiente ejemplo de un algoritmo de Python:

```
n ← 100
a ← 0
for k ← 1, n do
    a ← a + k
end for
write(a)
```

```
1 n = 100 # O(1)
2 a = 0 # O(1)
3 for k in range(n): # O(n)
4     a += k # n x O(1)
5 print(a) # O(1)
```

En este algoritmo, el comando $+=$ es un operador de asignación cuyo resultado se almacena en la variable a (es equivalente a la operación $a = a + k$). Utilizando la regla de la suma aritmética, se obtiene:

$$\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(n) + n \times \mathcal{O}(1) + \mathcal{O}(1)$$

La anterior representación se puede transformar en:

$$\mathcal{O}(1 + 1 + n + n + 1) = \mathcal{O}(3 + 2n) \approx \mathcal{O}(n)$$

Veamos otro ejemplo de análisis con el siguiente algoritmo de Python:

```

Lista ← False
n ← 100
x ← 50
for k ← 1, n do
    if k = x then
        Lista ← True
        break
    end if
end for
write(k, Lista)

```

```

1 Lista = False # O(1)
2 n = 100 # O(1)
3 x = 50 # O(1)
4 for k in range(n): # O(n)
5     if k == x: # n x O(1)
6         Lista = True # n x O(1)
7         break # n x O(1)
8 print(k, Lista) # O(1)

```

En este algoritmo, el comando *break* permite detener un ciclo o bucle por completo cuando se cumple una condición externa de una sentencia. Analizando el costo computacional, se obtiene el siguiente resultado:

$$\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(n) + n \times \mathcal{O}(1) + n \times \mathcal{O}(1) + n \times \mathcal{O}(1) + \mathcal{O}(1)$$

Esta operación es equivalente a:

$$\mathcal{O}(4) + \mathcal{O}(n) + 3n \times \mathcal{O}(1) \approx \mathcal{O}(n)$$

2.2 Análisis de bucles anidados, tiempo logarítmico y log-lineal

Cuando se construye un algoritmo con bucles anidados con ciclos *for* o con *while*, es necesario conocer varios casos para estimar el costo computacional. Veamos el primer caso de bucles anidados:

```

n ← 50
i ← 0
j ← 0
j ← 0
while i ≤ n do
    while j ≤ n do
        j ← j + 1
    end while
    k ← k + 1
    i ← i + 1
end while
write(i, j, k)

```

```

1 n = 50 # O(1)
2 i = 0 # O(1)
3 j = 0 # O(1)
4 k = 1 # O(1)
5 while i <= n: # O(n + 1)
6     while j <= n: # O(n(n + 1))
7         j = j + 1 # O(n*n)
8         k = k + 1 # O(n)
9         i = i + 1 # O(n)
10 print(i, j, k) # O(1)

```

En este algoritmo, la líneas 5, 8 y 9 se ejecutan n veces dentro del primer bucle *while*. Mientras tanto, la línea 6 se observa que el valor $n + 1$ se ejecuta n veces, es decir, $n \cdot (n + 1) = n^2 + n$, así

como la línea 7, que se ejecuta el valor n por n veces, es decir, $n \cdot n = n^2$. Al aplicar la regla de la suma aritmética, se obtiene:

$$\mathcal{O}(6) + \mathcal{O}(4n) + \mathcal{O}(2n^2) = \mathcal{O}(2n^2 + 4n + 6) \approx \mathcal{O}(n^2)$$

El anterior resultado indica un tiempo polinómico *cuadrático*, lo cual indica que dos bucles *while* anidados incrementa bastante el costo computacional.

Ahora analicemos el segundo caso de bucles:

```

 $n \leftarrow 100$ 
 $m \leftarrow 80$ 
 $i \leftarrow 0$ 
 $j \leftarrow 0$ 
while  $i \leq n$  do
     $i \leftarrow i + 1$ 
end while
while  $j \leq m$  do
     $j \leftarrow j + 1$ 
end while
write( $i, j$ )

```

```

1 n = 100 # O(1)
2 m = 80 # O(1)
3 i = 0 # O(1)
4 j = 0 # O(1)
5 while i <= n: # O(n + 1)
6     i = i + 1 # O(n)
7 while j <= m: # O(m + 1)
8     j = j + 1 # O(m)
9 print(i, j) # O(1)

```

En este algoritmo, los dos bucles son *independientes* puesto que el primer *while* representa $n+1$ veces mientras que el segundo *while* representa $m+1$, siendo $n \neq m$. Por consiguiente, el costo computacional esta dado por:

$$\mathcal{O}(7) + \mathcal{O}(2n) + \mathcal{O}(2m) \approx \mathcal{O}(n + m)$$

El anterior resultado indica que el costo computacional es *lineal*.

Cambiando de escenario de análisis, nos enfocaremos en el análisis de algoritmos con costo computacional logarítmico y log-lineal. Veamos el siguiente algoritmo:

```

 $n \leftarrow 64$ 
 $k \leftarrow 0$ 
while  $n > 1$  do
     $n \leftarrow n/2$ 
     $k \leftarrow k + 1$ 
end while
write( $n, k$ )

```

```

1 n = 64 # O(1)
2 k = 0 # O(1)
3 while n > 1: # O(log_{2}(n) + 1)
4     n = n/2 # O(log_{2}(n))
5     k = k + 1 # O(log_{2}(n))
6 print(n, k) # O(1)

```

Para este algoritmo, el bucle *while* tiene un costo computacional *logarítmico* por la siguiente deducción:

n	$\frac{n}{2}$	k
64	32	0
32	16	1
16	8	2
8	4	3
4	2	4
2	1	5

Table 1: Análisis del costo computacional logarítmico

Si se analiza la secuencia que nos muestra la Tabla 1, se observa que su comportamiento esta relacionado a un logarítmico. Por definición, un logaritmo es la potencia a la que hay que elevar un número para obtener otro número. Por ejemplo, $\log_{10}100 = 2$ ya que $10^2 = 100$ o a su vez, se analiza que:

$$\frac{100}{10} = \mathbf{10}$$

$$\frac{\mathbf{10}}{10} = 1$$

Observe que se dividió 100 en 2 oportunidades por 10 para obtener cociente igual a 1. Por consiguiente, la base 10 de 100 es **2**. Siguiendo la misma estrategia, en la Tabla 1 es evidente que sigue una secuencia 2^k , siendo k un valor entero positivo:

$$2^0 = 1$$

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^5 = 32$$

Por consiguiente, es evidente que se debe usar la base **2**:

$$\frac{64}{2} = 32$$

$$\frac{32}{2} = 16$$

$$\frac{16}{2} = 8$$

$$\frac{8}{2} = 4$$

$$\frac{4}{2} = 2$$

$$\frac{2}{2} = 1$$

Teniendo en cuenta lo anterior, si calculamos $\log_2 64 = 6$, el cual indica que se ha ejecutado **seis (6)** veces el bucle *while* (es decir, cuando k toma valores $\{0, 1, 2, 3, 4, 5\}$). Esta deducción confirma que el bucle *while* para este algoritmo es $\log_2(n)$ y por supuesto, el costo computacional es expresado como:

$$\mathcal{O}(4) + \mathcal{O}(3\log_2(n)) \approx \mathcal{O}(\log_2(n))$$

Nota: Es importante considerar que $\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(\log_2(n) + 1) + \mathcal{O}(\log_2(n)) + \mathcal{O}(\log_2(n)) + \mathcal{O}(1) = \mathcal{O}(4) + \mathcal{O}(3\log_2(n))$

Veamos ahora otro caso de complejidad computacional:

```

n ← 256
i ← 0
k ← 0
while i ≤ n do
    j ← n
    while j > 0 do
        j ← j/2
    end while
    i ← i + 1
    k ← k + i
    write(k)
end while
write(j)

```



```

1 n = 256 # O(1)
2 i = 0 # O(1)
3 k = 0 # O(1)
4 while i <= n: # O(n + 1)
5     j = n # O(n)
6     while j > 0: # O(n(log_2(n) + 1))
7         j = j/2 # O(n(log_2(n)))
8         i = i + 1 # O(n)
9         k = k + i # O(n)
10    print(k) # O(n)
11 print(j) # O(1)

```

En este algoritmo, hay dos bucles *while* anidados donde las líneas 4, 5, 8, 9 y 10 se ejecuta con un costo $\mathcal{O}(n)$ pero las líneas 6 y 7 se ejecutan en un tiempo Log Lineal. En este caso, el segundo *while* indica que se ejecuta n veces un tiempo $\log_2(n) + 1$ y la línea 7 se ejecuta n veces un tiempo $\log_2(n)$. A partir de este análisis, el costo computacional del algoritmo es:

Línea 1:	$\mathcal{O}(1)$
Línea 2:	$\mathcal{O}(1)$
Línea 3:	$\mathcal{O}(1)$
Línea 4:	$\mathcal{O}(n) + \mathcal{O}(1)$
Línea 5:	$\mathcal{O}(n)$
Línea 6:	$\mathcal{O}(n \log_2(n)) + \mathcal{O}(n)$
Línea 7:	$\mathcal{O}(n \log_2(n))$
Línea 8:	$\mathcal{O}(n)$
Línea 9:	$\mathcal{O}(n)$
Línea 10:	$\mathcal{O}(1)$
Conteo:	$\mathcal{O}(2n \log_2(n)) + \mathcal{O}(5n) + \mathcal{O}(5)$

Por consiguiente, si consideramos el costo computacional como $\mathcal{O}(2n \log_2(n) + 5n + 5)$ entonces podemos aproximar a $\mathcal{O}(n \log_2(n))$ o *log-linear*. Para validar esta afirmación, podemos comparar $\mathcal{O}(2n \log_2(n))$ con respecto a $\mathcal{O}(5n)$ para valores $1 \leq n \leq 256$. Como se observa en la Fig. 3, se concluye que $\mathcal{O}(5n) \leq \mathcal{O}(2n \log_2(n))$ y por consiguiente, $\mathcal{O}(2n \log_2(n)) + \mathcal{O}(5n) + \mathcal{O}(5) \approx \mathcal{O}(n \log_2(n))$.

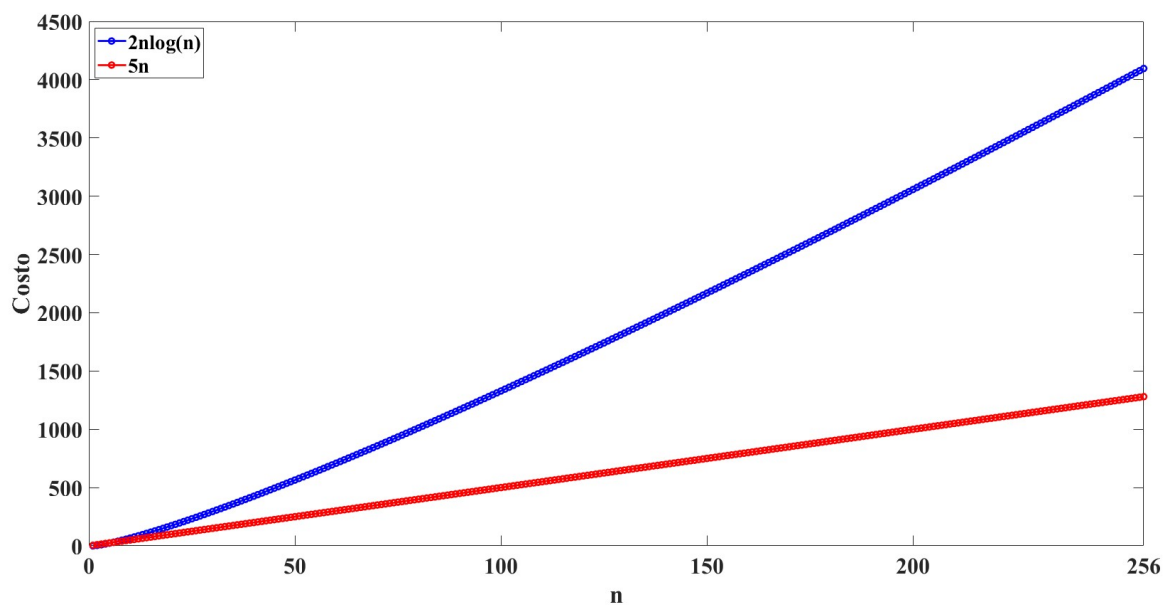


Figure 3: Gráfica de $\mathcal{O}(2n \log_2(n))$ vs $\mathcal{O}(5n)$

3 Miscelánea de Análisis Algorítmico

3.1 Análisis para el algoritmo 1

Algoritmo 1:

```
 $n \leftarrow 256$   
 $k \leftarrow 0$   
 $i \leftarrow 1$   
while  $i \leq n$  do  
     $k \leftarrow k + i$   
     $i \leftarrow i + 1$   
end while  
 $write(k)$ 
```

```
1 n = 256 # O(1)  
2 k = 0 # O(1)  
3 i = 1 # O(1)  
4 while i <= n: # O(n + 1)  
5     k = k + i # O(n)  
6     i = i + 1 # O(n)  
7 print(k) # O(1)
```

Determinando el costo computacional:

$$\mathcal{O}(5) + \mathcal{O}(3n) \approx \mathcal{O}(n)$$

Hasta este punto, el algoritmo tiene un costo $\mathcal{O}(n)$ pero es importante analizar la línea 5. Como se puede apreciar, $k = k + i$ se repite para n veces, lo cual genera un comportamiento cuadrático, tal y como se observa en la Fig. 4. A partir de este análisis, se puede afirmar que $k = k + i$ se comporta como una *serie geométrica*:

- Si $i = 1$, entonces $k = 0 + 1 = \mathbf{1}$
- Si $i = 2$, entonces $k = \mathbf{1} + 2 = \mathbf{3}$
- Si $i = 3$, entonces $k = \mathbf{3} + 3 = \mathbf{6}$
- Si $i = 4$, entonces $k = \mathbf{6} + 4 = \mathbf{10}$
- ...
- Si $i = 256$, entonces $k = 32640 + 256 = \mathbf{32896}$

Lo anterior es equivalente a:

- Si $i = 1$, entonces $k = \frac{1(1+1)}{2} = 1$
- Si $i = 2$, entonces $k = \frac{2(2+1)}{2} = 3$
- Si $i = 3$, entonces $k = \frac{3(3+1)}{2} = 6$
- Si $i = 4$, entonces $k = \frac{4(4+1)}{2} = 10$
- ...
- Si $i = 256$, entonces $k = \frac{256(256+1)}{2} = \mathbf{32896}$

Por consiguiente, el algoritmo 1 se puede reescribir como:

```
 $n \leftarrow 256$   
 $k \leftarrow 0$   
 $i \leftarrow 1$   
while  $i \leq n$  do  
     $k \leftarrow i * (i + 1) / 2$   
end while  
 $write(k)$ 
```

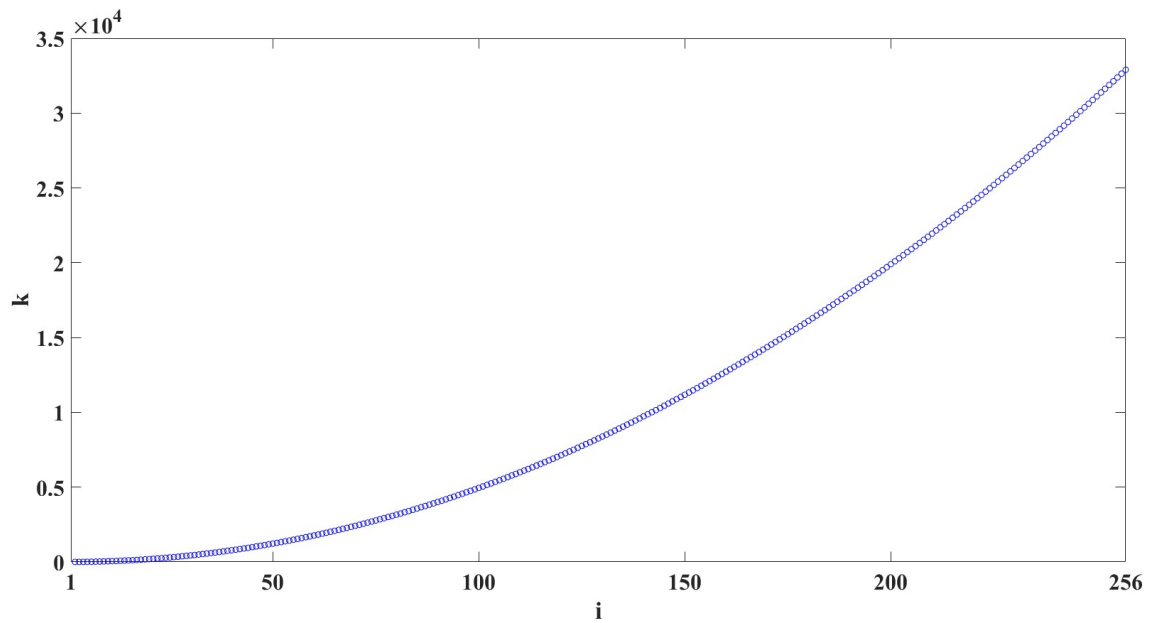


Figure 4: Gráfica de $k = k + 1$

```

1 n = 256
2 k = 0
3 i = 1
4 while i <= n:
5     k = (i*(i + 1))/2
6 print(k)

```

Conociendo que el resultado final del algoritmo es el último valor que se calcula de k cuando $i = 256$, se puede reducir el costo computacional $\mathcal{O}(n)$ a $\mathcal{O}(1)$ si se reescribe el algoritmo como:

```

n ← 256
k ← n * (n + 1) / 2
write(k)

```

```

1 n = 256 # O(1)
2 k = (n*(n + 1))/2 # O(1)
3 print(k) # O(1)

```

3.2 Análisis para el algoritmo 2

Algoritmo 2:

```

n ← 256
k ← 0
i ← 1
j ← 0
while i ≤ n do
    while j ≤ i do
        k ← k + j
        j ← j + 1
    end while
    i ← i + 1
end while
write(k)

```

```

1 n = 256
2 k = 0
3 i = 1
4 j = 0
5 while i <= n:
6     while j <= i:
7         k = k + j
8         j = j + 1
9     i = i + 1
10 print(k)

```

En el Algoritmo 2, el segundo bucle *while* se comporta exactamente como la serie geométrica del Algoritmo 1, tal y como se observa en la línea 7 del código (por supuesto, la línea 8 también genera un comportamiento como una serie geométrica). Teniendo en cuenta esta afirmación, el costo computacional se calcula a continuación:

```

1 n = 256 # O(1)
2 k = 0 # O(1)
3 i = 1 # O(1)
4 j = 0 # O(1)
5 while i <= n: # O(n + 1)
6     while j <= i: # O(n(n + 1)/2 + n)
7         k = k + j # O(n(n + 1)/2)
8         j = j + 1 # O(n(n + 1)/2)
9     i = i + 1 # O(n)
10 print(k) # O(1)

```

$$\mathcal{O}(6) + \mathcal{O}(3n) + \mathcal{O}\left(\frac{3}{2}n(n+1)\right) = \mathcal{O}\left(\frac{3}{2}n^2 + \frac{9}{2}n + \frac{15}{2}\right) \approx \mathcal{O}(n^2)$$

El Algoritmo 2 genera un tiempo polinómico *cuadrático* en el peor de los casos.

4 Referencias

- Roberto Flórez Rueda. *Algoritmia Básica*, Segunda Edición, Universidad de Antioquia, 2011.
- Thomas Mailund. *Introduction to Computational Thinking: Problem Solving, Algorithms, Data Structures, and More*, Apress, 2021.